FE621 HW2

Cian Gahan

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from FE621.pricing.tree import BinomialTree, TrinomialTree
from FE621.pricing.fdm import ExplicitFD, ImplicitFD, CrankNicolsonFD
from FE621.pricing.black_scholes import BlackScholes
```

Problem 1: The Binomial Tree

a) Construct code to calculate option values using an additive binomial tree. For this part you need four versions: European and American as well as Call and Put. You may use the same tree construction (and function) for all options.

- See `src/FE621/pricing/tree.py` for binomial tree implementation

b) Download Option prices (you can use the Bloomberg Terminal, Yahoo! Finance, etc.) for an equity, for 3 different maturities (1 month, 2 months, and 3 months) and 20 strike prices close to the value at the money. If 3 months does not exist use next one available. Please download the data DURING THE TRADING DAY (9:00am to 4:30pm ET). Otherwise your values will be way off. Do not forget to download the value of the underlying. For each strike price in the data, use the implied vol values in Homework 1 (see Problem 1c) and the current short-term interest rate (the Feds Fund rate is a good choice). Calculate the option price (European Calls and Puts) using the binomial tree, and compare the results with the Black–Scholes price. Use at least 200 steps in your tree construction. Treat the options as American as well and plot these values side by side with the European and Black Scholes values. When you create the plot do not forget to plot the bid-ask values as well. If you downloaded DATA 2 for last assignment you can use that as your data set.

- I used DATA2 as my dataset and imported csv from HW1 containing day 2 options with day 1 implied vols. Filtering for moneyness between 1.06 and 0.94 yields 20 unique strike prices

In [2]:
```
# Using DATA2 from Homework 1 as dataset and TSLA as equity
data2 = pd.read_csv("../data/cleaned/data2_with_iv.csv", index_col=0)
data2_tsla = data2[(data2["underlying"] == "TSLA") &
                   (data2["moneyness"] < 1.06) &
                   (data2["moneyness"] > 0.94)]
data2_tsla.head()
```

Out[2]:

	contractSymbol	strike	bid	ask	optionType	expiration	underlying
192	TSLA260220C00397500	397.5	25.85	26.10	call	2026-02-20	TSLA
193	TSLA260220C00400000	400.0	23.55	23.85	call	2026-02-20	TSLA
194	TSLA260220C00402500	402.5	21.50	21.65	call	2026-02-20	TSLA
195	TSLA260220C00405000	405.0	19.60	19.80	call	2026-02-20	TSLA
196	TSLA260220C00407500	407.5	17.65	17.85	call	2026-02-20	TSLA

In [3]:

```
def row_binomial_price(row):
    if row["optionType"] == "call":
        tree = BinomialTree(
            np.log1p(row["fedFunds"]),
            row["impliedVolatility"],
            row["underlyingPrice"],
            row["daysToMaturity"] / 365,
            300
        )
        return (
            tree.price_option(row["strike"], call=True, american=False),
            tree.price_option(row["strike"], call=True, american=True)
        )
    else:
        tree = BinomialTree(
            np.log1p(row["fedFunds"]),
            row["impliedVolatility"],
            row["underlyingPrice"],
            row["daysToMaturity"] / 365,
            300
        )
        return (
            tree.price_option(row["strike"], call=False, american=False),
            tree.price_option(row["strike"], call=False, american=True)
        )

result = data2_tsla.apply(row_binomial_price, axis=1)
res_df = pd.DataFrame(result.to_list(), index=data2_tsla.index, columns=["eu
data2_tsla = pd.concat([data2_tsla, res_df], axis=1)
data2_tsla.head()
```

Out[3]:

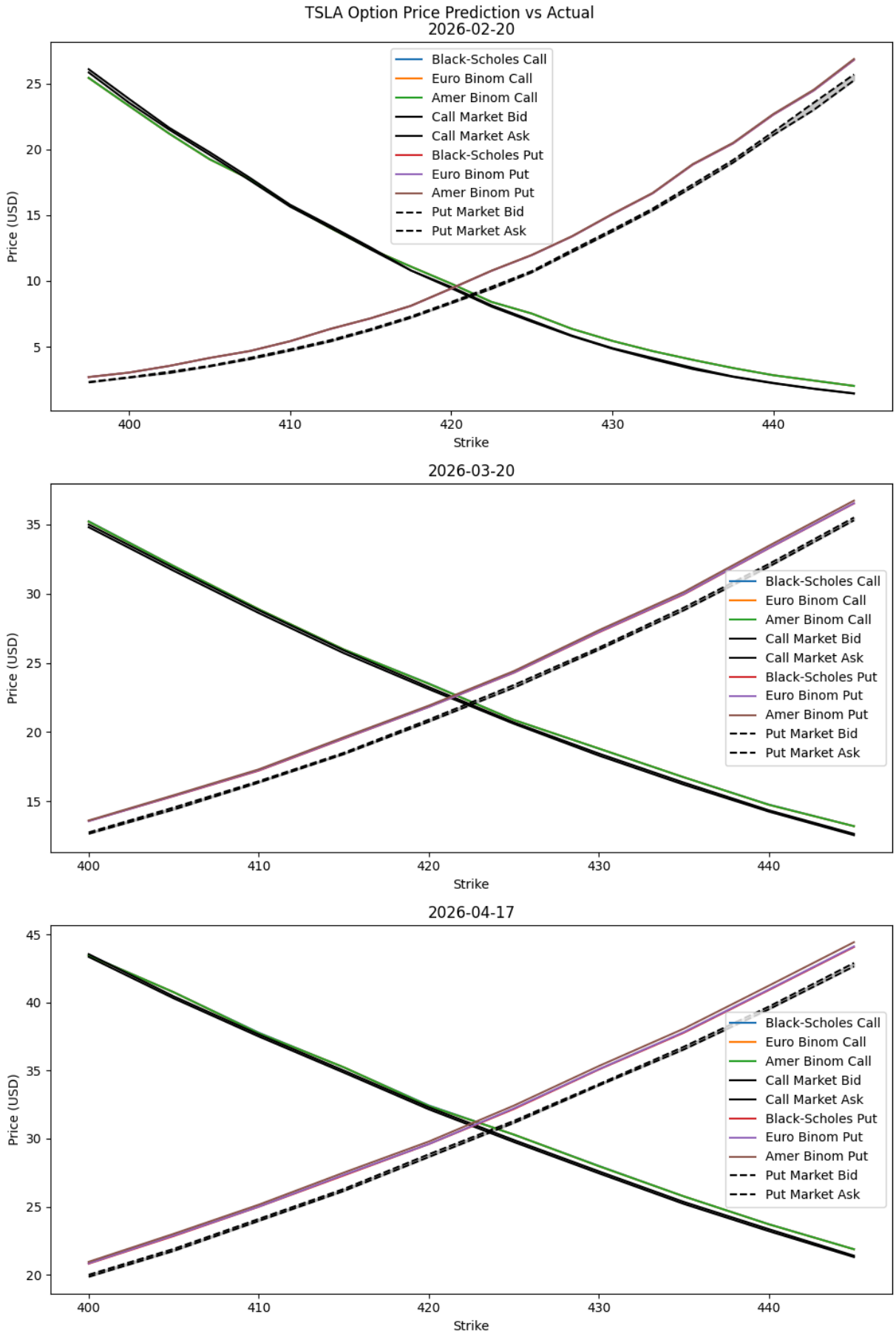
	contractSymbol	strike	bid	ask	optionType	expiration	underlying
192	TSLA260220C00397500	397.5	25.85	26.10	call	2026-02-20	TSLA
193	TSLA260220C00400000	400.0	23.55	23.85	call	2026-02-20	TSLA
194	TSLA260220C00402500	402.5	21.50	21.65	call	2026-02-20	TSLA
195	TSLA260220C00405000	405.0	19.60	19.80	call	2026-02-20	TSLA
196	TSLA260220C00407500	407.5	17.65	17.85	call	2026-02-20	TSLA

5 rows × 21 columns

In [4]:

```
fig, axs = plt.subplots(nrows=3, figsize=(10, 15))

for i, expiration in enumerate(["2026-02-20", "2026-03-20", "2026-04-17"]):
    calls = data2_tsla[(data2_tsla["optionType"] == "call") & (data2_tsla["e
    puts = data2_tsla[(data2_tsla["optionType"] == "put") & (data2_tsla["exp
    axs[i].plot(calls["strike"], calls["calcprice"], label="Black-Scholes Ca
    axs[i].plot(calls["strike"], calls["euroBinomPrice"], label="Euro Binom
    axs[i].plot(calls["strike"], calls["amerBinomPrice"], label="Amer Binom
    axs[i].plot(calls["strike"], calls["bid"], label="Call Market Bid", colo
    axs[i].plot(calls["strike"], calls["ask"], label="Call Market Ask", colo
    axs[i].fill_between(calls["strike"], calls["bid"], calls["ask"], color="
    axs[i].plot(puts["strike"], puts["calcprice"], label="Black-Scholes Put"
    axs[i].plot(puts["strike"], puts["euroBinomPrice"], label="Euro Binom Pu
    axs[i].plot(puts["strike"], puts["amerBinomPrice"], label="Amer Binom Pu
    axs[i].plot(puts["strike"], puts["bid"], "k--", label="Put Market Bid")
    axs[i].plot(puts["strike"], puts["ask"], "k--", label="Put Market Ask")
    axs[i].fill_between(puts["strike"], puts["bid"], puts["ask"], color="gra
    axs[i].set_xlabel("Strike")
    axs[i].set_ylabel("Price (USD)")
    axs[i].legend()
    axs[i].set_title(expiration)
fig.suptitle("TSLA Option Price Prediction vs Actual")
fig.tight_layout()
plt.show()
```


TSLA Option Price Prediction vs Actual
2026-02-20
Black-Scholes Call
Euro Binom Call
Amer Binom Call
Call Market Bid
Call Market Ask
Black-Scholes Put
Euro Binom Put
Amer Binom Put
Put Market Bid
Put Market Ask
Price (USD)
Strike
2026-03-20
Price (USD)
Strike
2026-04-17
Price (USD)
Strike

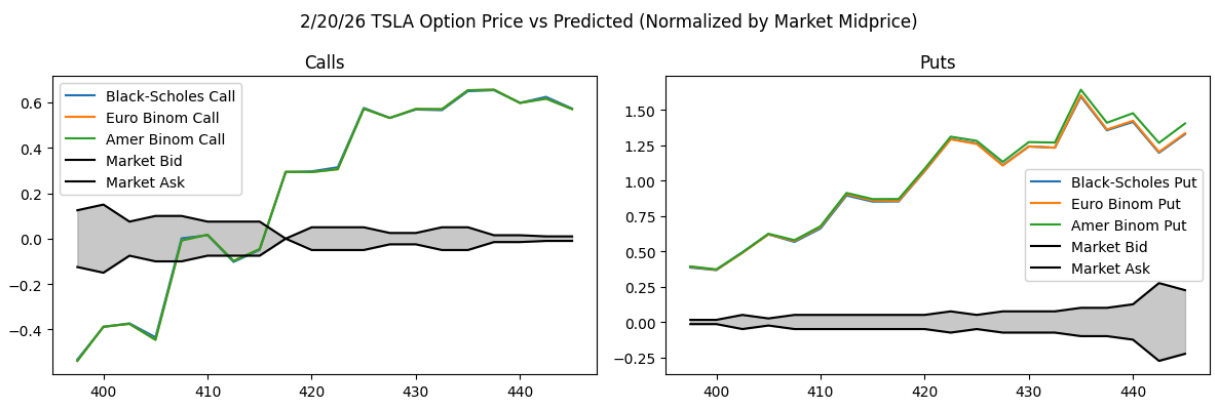
Given the zoomed-out nature of the chart I decided to make a second chart that normalizes everything by subtracting the market mid price of the options, using just one maturity as an example:

In [5]:
```
calls = data2_tsla[(data2_tsla["optionType"] == "call") & (data2_tsla["expir
puts = data2_tsla[(data2_tsla["optionType"] == "put") & (data2_tsla["expirat
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

axs[0].plot(calls["strike"], calls["calcprice"] - calls["midprice"], label="
axs[0].plot(calls["strike"], calls["euroBinomPrice"] - calls["midprice"], la
axs[0].plot(calls["strike"], calls["amerBinomPrice"] - calls["midprice"], la
axs[0].plot(calls["strike"], calls["bid"] - calls["midprice"], label="Market
axs[0].plot(calls["strike"], calls["ask"] - calls["midprice"], label="Market
axs[0].fill_between(calls["strike"], calls["bid"] - calls["midprice"], calls
axs[0].legend()
axs[0].set_title("Calls")

axs[1].plot(puts["strike"], puts["calcprice"] - puts["midprice"], label="Bla
axs[1].plot(puts["strike"], puts["euroBinomPrice"] - puts["midprice"], label
axs[1].plot(puts["strike"], puts["amerBinomPrice"] - puts["midprice"], label
axs[1].plot(puts["strike"], puts["bid"] - puts["midprice"], label="Market Bi
axs[1].plot(puts["strike"], puts["ask"] - puts["midprice"], label="Market As
axs[1].fill_between(puts["strike"], puts["bid"] - puts["midprice"], puts["as
axs[1].legend()
axs[1].set_title("Puts")

fig.suptitle("2/20/26 TSLA Option Price vs Predicted (Normalized by Market M
fig.tight_layout()
plt.show()
```



c) Comment of the table in the previous part.

It seems that the Call prediction was better than the Put prediction, as the put prediction was seemingly always too high for each maturity in the data. Both the call and put are within the neighborhood of the market midprice. The call price does cross the bid-ask range in the closest maturity, but the put price does not. It also seems that as the put option maturity increases, the prediction gets slightly less accurate (as well as the call options, though not to the same degree). Perhaps the implied volatility of the put options

in the market experienced a correction in the second day of data that was not accounted for when using the prior days' volatility.

d) Consider $N \in \{10, 20, 30, 40, 50, 100, 150, 200, 250, 300, 350, 400\}$. Compute and plot the absolute error for the European Put ϵ_N as a function of $N \in \mathbb{N}*$ the number of steps in the tree:

$$\epsilon_N = |P^{BSM}(S_0, K, T, r; \sigma) - P^{BinomTree}(S_0, K, T, r; \sigma)|$$

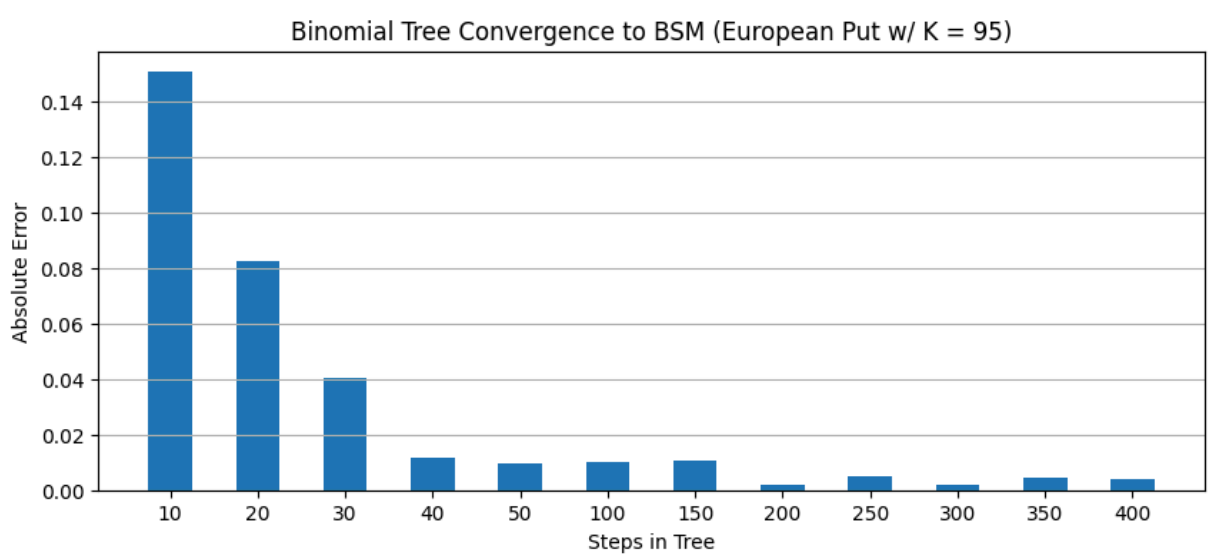
In [6]:
```
ns = [10, 20, 30, 40, 50, 100, 150, 200, 250, 300, 350, 400]

s0 = 100
K = 95
T = 1
r = 0.05
sigma = 0.2

bsm_price = BlackScholes.put(s0, K, T, r, sigma)
errors = []

for n in ns:
    tree = BinomialTree(r, sigma, s0, T, n)
    errors.append(abs(tree.price_option(K, call=False, american=False) - bsm

plt.figure(figsize=(10,4))
plt.bar(range(len(ns)), errors, width=0.5)
plt.xticks(range(len(ns)), labels=ns)
plt.grid(visible=True, axis="y")
plt.xlabel("Steps in Tree")
plt.ylabel("Absolute Error")
plt.title("Binomial Tree Convergence to BSM (European Put w/ K = 95)")
plt.show()
```



Problem 2: Pricing Exotic Options using Trinomial Trees

We will use here a synthetic example to illustrate using trees to price path dependent options.

a) Construct a trinomial tree to calculate the price of an European Up-and-Out call option. Use S0 = 10, strike K = 10, maturity T = 0.3, volatility σ = 0.2, short rate r = 0.01, dividends δ = 0 and barrier H = 11. See Chapter 3 in [1] for the trinomial tree construction. Use as many steps in your tree as you think are necessary. Hint. For the option pricing you may consult the algorithm in the book [1] and try and figure out how to modify the code there to work with the new option and the new tree. Note the book details the computation using a binomial tree while we ask you to use a trinomial tree here.

- See `src/FE621/pricing/tree.py` for Trinomial tree implementation

In [7]:
```
s0 = 10
K = 10
T = 0.3
sigma = 0.2
r = 0.01
H = 11

# Using 300 steps to keep order of convergence low (Δt = 0.001)
trinom_tree = TrinomialTree(r, sigma, s0, T, n=300)
up_out_call_price = trinom_tree.price_barrier_option_knock_out(K, H, call=Tr
print(up_out_call_price)
```

```
0.05831284403967179
```

b) For the European Up-and-Out Call option explicit formulas exist. For example, implement the formula (5.2) from [2] and compare your results with part (a). Use the same parameters as before. Are your results matching? Note the paper is uploaded to the course shell under the name “Pricing Barrier Options”

- See `src/FE621/pricing/black_scholes.py` for explicit formula implementation

In [8]:
```
print(BlackScholes.up_out_call(s0, K, H, T, r, sigma))
```

```
0.05309279660325303
```

The price is quite close but slightly lower than the tree approximation. This seems plausible given the order of convergence above is proportional to the actual error by an unknown constant (in this case an error of around 0.005).

c) Price an European Up-and-In call option, using the same parameters as before. Hint. Two methods can be employed: the analytical solution in (5.1) or the In-Out parity. Use both methods in order to verify your results.

- See `src/FE621/pricing/tree.py` for trinomial tree knock-in barrier option pricing implementation

In [9]:
```
print("Analytical/Tree-Based:", trinom_tree.price_barrier_option_knock_in(K,
print("In/Out Parity:", trinom_tree.price_option(K, call=True, american=Fals
```

```
Analytical/Tree-Based: 0.3926115418733819
In/Out Parity: 0.3926115418733819
```

d) Calculate the price of an AMERICAN Up and In Put option

- This is implemented identically to the european knock-in option inside `src/FE621/pricing/tree.py`, except within the hit state tree the node values are calculated with the possibility of early exercise

In [10]:
```
print(trinom_tree.price_barrier_option_knock_in(K, H, call=False, american=T
```

```
0.015251201994063806
```

Problem 3: Finite difference methods.

a) Implement the Explicit Finite Difference method to price both European Call and Put options. Hint. See Chapter 3 in [1].

b) Implement the Implicit Finite Difference method to price European Call and Put options.

c) Implement the Crank-Nicolson Finite Difference method to price both European Call and Put options.

- See `src/FE621/pricing/fdm.py` for finite difference method pricing implementations

In [11]:
```
S0 = 100
K = 100
sigma = 0.2
r = 0.06
T = 1
div = 0.0
steps = 1000

print("Black-Scholes:", BlackScholes.call(S0, K, T, r, sigma))

efd = ExplicitFD(r, div, sigma, S0, T, steps)
print("Explicit FD:", efd.price_option(K, call=True))


print("Implicit FD:", ImplicitFD.price_option(S0, K, r, div, sigma, T, N=100
print("Crank-Nicolson FD:", CrankNicolsonFD.price_option(S0, K, r, div, sigm
```

```
Black-Scholes: 10.989549152625997
Explicit FD: 10.987944816354654
Implicit FD: 10.988404816989247
Crank-Nicolson FD: 10.989460741541883
```

d) For both the Explicit and Implicit Finite Difference schemes estimate the numbers Δt, Δx as well as the total number N_j of points on the space grid x to obtain a desired error of $\epsilon = 0.001$. Hint. You need to solve this part in a theoretical way. Please use the convergence order as the actual error of the estimate.

The order of convergence (treated in this case as the error) is $\epsilon = \Delta x^2 + \Delta t$.

Since $\Delta x = \sigma\sqrt{3\Delta t}$, $\epsilon = \Delta t(1 + 3\sigma^2) = 0.001$

Therefore $\Delta t = \frac{0.001}{1+3\sigma^2}$, so the space grid width $N = 1000T + 3000T\sigma^2$

Explicit FD: Total points on the grid (that are used, the size of the 2d array would be $N(2N+1)$) is $N^2 = 1,000,000T^2 + 6,000,000T\sigma^2 + 9,000,000\sigma^4$

Implicit FD: $N(2N+1)$ points where $N = 1000T + 3000T\sigma^2$

e) Consider S0 = 100, K = 100, T = 1 year, $\sigma = 25\%$, r = 6%, $\delta = 0.03$. Calculate and report the price for European Call and Put using explicit, implicit FD and Crank-Nicolson methods and the number of steps that you calculated in the previous point (part d).

In [12]:
```
S0 = 100
K = 100
sigma = 0.25
r = 0.06
T = 1
div = 0.03
steps = int(1000 * T + 3000 * T * sigma ** 2) + 1
print("Steps:", steps)

print("Call:")
efd = ExplicitFD(r, div, sigma, S0, T, steps)
print("Explicit FD:", efd.price_option(K, call=True))
print("Implicit FD:", ImplicitFD.price_option(S0, K, r, div, sigma, T, N=100
print("Crank-Nicolson FD:", CrankNicolsonFD.price_option(S0, K, r, div, sigm

print("Put:")
print("Explicit FD:", efd.price_option(K, call=False))
print("Implicit FD:", ImplicitFD.price_option(S0, K, r, div, sigma, T, N=100
print("Crank-Nicolson FD:", CrankNicolsonFD.price_option(S0, K, r, div, sigm
```

```
Steps: 1188
Call:
Explicit FD: 11.011409899773025
Implicit FD: 11.011560396068914
Crank-Nicolson FD: 11.013005856503359
Put:
Explicit FD: 8.143167270380223
Implicit FD: 8.143585311456514
Crank-Nicolson FD: 8.144904928587046
```

f) Repeat part (d) of this problem but this time get the empirical number of iterations. Specifically, obtain the Black Scholes price for the data in (e), then do an iterative procedure to figure out the Δx, Δt, N , and Nj to obtain an accuracy of $\epsilon = 0.001$. Put the results of the 3 methods (EFD, IFD, CNFD) side by side in a table and write your observations.

In [21]:
```
S0 = 100
K = 100
sigma = 0.25
r = 0.06
T = 1
div = 0.03

bsm_price = BlackScholes.div_call(S0, K, T, r, div, sigma)

explicit_error = float("inf")
explicit_steps = 100

while explicit_error > 0.001:
    print(explicit_steps, explicit_error)
    efd = ExplicitFD(r, div, sigma, S0, T, explicit_steps)
    explicit_error = abs(efd.price_option(K, call=True) - bsm_price)
    explicit_steps += 100

print(explicit_steps)
print(explicit_error)

implicit_error = float("inf")
implicit_steps = 100

while implicit_error > 0.001:
    print(implicit_steps, implicit_error)
    implicit_steps += 100
    implicit_price = ImplicitFD.price_option(
        S0, K, r, div, sigma, T,
        N=implicit_steps,
        Nj=implicit_steps,
        dx=2 / implicit_steps,
        call=True,
        american=False
    )
    implicit_error = abs(implicit_price - bsm_price)

print(implicit_steps)
```

```
print(implicit_error)

cn_error = float("inf")
cn_steps = 100

while cn_error > 0.001:
    print(cn_steps, cn_error)
    cn_steps += 100
    cn_price = CrankNicolsonFD.price_option(
        S0, K, r, div, sigma, T,
        N=cn_steps,
        Nj=cn_steps,
        dx=2 / implicit_steps,
        call=True,
        american=False
    )
    cn_error = abs(cn_price - bsm_price)

print(cn_steps)
print(cn_error)
```

```
100 inf
200 0.019863663817051957
300 0.00992113945582318
400 0.006611739297760266
500 0.004957924946070946
600 0.003965918571990912
700 0.0033046983138440567
800 0.002832455507364884
900 0.002478304749425675
1000 0.0022028727198097897
1100 0.001982538782019816
1200 0.0018022732854596768
1300 0.0016520573450282683
1400 0.0015249553154497164
1500 0.0014160134667520197
1600 0.0013215992519661768
1700 0.0012389883730499207
1800 0.0011660976315930327
1900 0.0011013068156575656
2000 0.0010433368454911829
2100
0.0009911644838087597
100 inf
200 0.00905571274080863
300 0.0056298384075805785
400 0.0040698523564994815
500 0.0031827197926972417
600 0.0026116377364910903
700 0.00221367876709877
800 0.001920652605175377
900 0.0016959680269437172
1000 0.0015182514703671046
1100 0.001374189705236617
1200 0.0012550611903527908
1300 0.0011549153766434728
1400 0.0010695542376506495
1500
0.0009959314036525058
100 inf
200 0.33093257582766356
300 0.021906635206740077
400 0.002941394481934978
500
0.00047829616007177833
```

In [25]:

```
results = pd.DataFrame({
    "dt": (1 / explicit_steps, 1 / implicit_steps, 1 / cn_steps),
    "dx": (2 / explicit_steps, 2 / implicit_steps, 2 / cn_steps),
    "error": (explicit_error, implicit_error, cn_error)
})
results.index = ["explicit", "implicit", "crank-nicolson"]
results
```

Out[25]:

	dt	dx	error
explicit	0.000476	0.000952	0.000991
implicit	0.000667	0.001333	0.000996
crank-nicolson	0.002000	0.004000	0.000478

I imagine my iterative procedure could be improved on as it was rather crude. However, the general pattern of convergence being Crank-Nicolson fastest, followed by implicit then explicit, checks out, confirming prior understanding

Bonus Problem 2: Pricing a Chooser Option.

A Chooser option is a derivative product which gives the right to the holder to own at maturity either a Put Option or a Call Option depending on the relationship of the Stock price at maturity and the strike price K. In other words the option payoff is determined as:

$$\mathbb{E}[(S_T - K)_+ \mathbf{1}_{\{S_T \geq K\}} + (K - S_T)_+ \mathbf{1}_{\{S_T < K\}}]$$

a) Develop a binomial tree model to price a chooser option on a stock with exercise price $K = 100$, $S_0 = 100$, maturity $T = 0.3$, volatility $\sigma = 0.15$. Assume a risk-free rate is $r = 0.01$.

- See `src/FE621/pricing/tree.py` for binomial tree pricing implementation (`BinomialTree.price_chooser_option`)

In [14]:
```
chooser_tree = BinomialTree(r_f = 0.01, sigma=0.15, s0=100, T=0.3, n=100) #
print(chooser_tree.price_chooser_option(K=100, american=False))
```

```
6.531642686788443
```

b) Describe how an American type Chooser Option would differ from the above Option. Extend your algorithm to approximate the price of such an option.

Assuming the Chooser optionality extends backward as with early exercise, that is, one can choose to exercise the option early and choose which type (call/put) the payoff is selected from, this functions similarly to an American call or put, just with a different payoff diagram. The difference between this and the European version is the early exercise, but the functional payoff diagram of $|S_T - K|$ remains the same.

The `price_chooser_option` function above includes an `american` boolean flag which incorporates early exercise into the pricing:

In [15]:
```
print(chooser_tree.price_chooser_option(K=100, american=True))
```

```
6.5500853673803165
```

We can see that the value of the option increases slightly when it becomes American, which makes sense as the option's put leg could plausibly create conditions where early exercise is optimal.